

App Dev Project Report

1. Student Details

Name: Aman Gupta

Roll Number: DS24f2002969

Email: 24f2002969@ds.study.iitm.ac.in

About Me: I am Aman Gupta, a student with a strong interest in web application development and data-driven technologies. I enjoy learning by building meaningful applications that combine analytics, user experience, and practical problem-solving. I am also currently serving as Deputy Secretary in Nilgiri House.

2. Project Details

Project Title: SuperSet - Placement Portal Application V1

Problem Statement:

This project implements a **Flask-based Placement Portal** designed to support the placement process for students and companies. The system provides **role-based access control** for three types of users:

- Admin
- Company
- Student

The platform allows companies to post placement drives, students to browse and apply for job opportunities, and administrators to monitor and approve company registrations and job postings.

The portal supports workflows such as **company approval, job posting moderation, student job applications, and application status tracking**. The system is designed to be modular, responsive, and easy to maintain while demonstrating the core concepts of modern web application development.

Approach:

I approached the problem by designing a role-based workflow that mirrors the real placement process and enforcing it end-to-end: admins validate companies and drives, companies manage applicants through a defined status pipeline, and

students see only

approved opportunities and can track their application history. The backend enforces these rules with filtered queries, approval flags, and status logging; the frontend surfaces them through role-specific dashboards and timelines so each stakeholder gets a focused, reliable experience.

3. AI/LLM Declaration

I used ChatGPT(GPT-5) & Claude(Anthropic) as an engineering assistant for selected tasks, mainly:

- modular refactoring of backend structure (splitting routes into Blueprints)
- schema migration/backfill logic planning (status-history support)
- improving chart configuration and responsive UI consistency,
- documentation/report structuring and API description formatting.

Estimated AI/LLM contribution is **15–20%**, focused on architecture-level suggestions, code cleanup patterns, and documentation quality.

Core implementation decisions, validation rules, business logic, integration, testing, and final debugging were performed manually by me.

4. Technologies and Frameworks Used

The project uses the following technologies:

Python 3 - Core backend language for all application logic.

Flask - Web framework for routing, sessions, HTTP handling, and Blueprint organisation.

Flask-SQLAlchemy - ORM for defining models and managing table relationships.

SQLite - Lightweight database for all application data.

Jinja2 - Server-side HTML templating with role-specific base layouts.

Bootstrap 5 - Responsive UI components and grid layout.

Chart.js - Pie and bar charts on Admin, Company, and Student dashboards.

Werkzeug - Password hashing and secure PDF resume uploads via `secure_filename`

5. Database Schema / ER Diagram

The database has six tables. All relationships use foreign keys enforced by SQLAlchemy.

User - Base authentication entity. Fields: id, username (unique), password, role (admin/company/student), is_blacklisted. Passwords are hashed using Werkzeug.

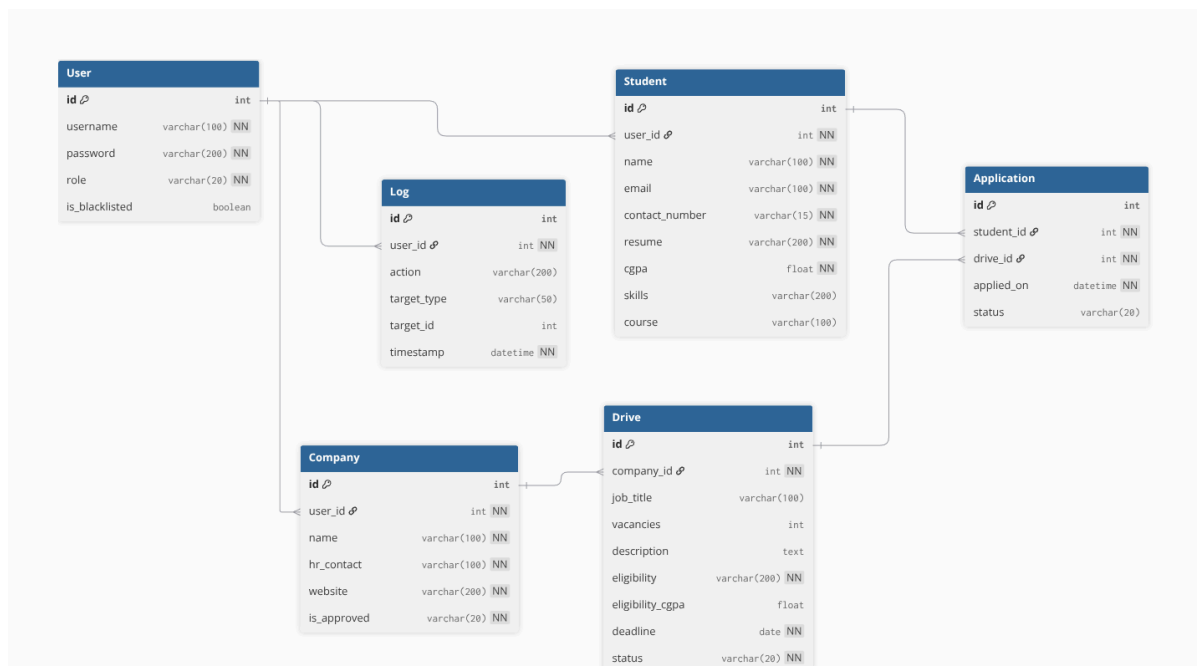
Company - Company profile linked to one User. Fields: id, user_id (FK), name, hr_contact, website, is_approved (pending/approved/rejected). One Company -> many Drives.

Student - Student profile linked to one User. Fields: id, user_id (FK), name, email (unique), contact_number, cgpa, skills, course, resume (path to uploaded PDF). One Student -> many Applications.

Drive - Placement drive posted by a company. Fields: id, company_id (FK), job_title, vacancies, description, eligibility, eligibility_cgpa, deadline, status (pending/active/closed/rejected). One Drive -> many Applications.

Application - Student-Drive mapping. Fields: id, student_id (FK), drive_id (FK), applied_on, status (applied/shortlisted/placed/rejected/withdrawn). UNIQUE constraint on (student_id, drive_id) prevents duplicate applications.

Log - Universal audit log. Fields: id, user_id (FK), action, target_type, target_id, timestamp. Every action across all roles is recorded here.



6. API Resource Endpoints

The application exposes **36 routes** across **4 Role-based Blueprints**.

Public/Auth - / (home), /login, /logout, /register/student, /register/company

Admin (/admin/...) - dashboard, students, companies, drives, application, logs, student/view/<id>, student/edit/<id>, company/view/<id>, company/edit/<id>, blacklist/<id> (POST), company/approve/<id> (POST), company/reject/<id> (POST), drive/approve/<id> (POST), drive/reject/<id> (POST), drive/close/<id> (POST)

Company (/company/...) - dashboard, profile, profile/edit, drive, drive/create, drive/view/<id>, drive/edit/<id>, drive/close/<id> (POST), applications, application/view/<id>, application/shortlist/<id> (POST), application/place/<id> (POST), application/reject/<id> (POST), notifications

Student (/student/...) - dashboard, profile, profile/edit, drives, drive/view/<id>, drive/apply/<id> (POST), applications, application/view/<id>, application/withdraw/<id> (POST), notifications

7. Architecture and Features

The application follows a **modular Flask architecture**.

The main file **app.py** handles application configuration, database initialization, and route registration.

The project structure is organized as follows:

- **route/** - Contains route handlers grouped by functionality (auth, admin, company, student, public, error)
- **model.py** - Contains all SQLAlchemy database model definitions
- **extra_services.py** - Contains the ensure_schema() helper that runs live ALTER TABLE migrations on startup to keep older local databases compatible
- **templates/** - Contains Jinja2 templates organised into role-specific subfolders, each with their own base.html
- **static/** - Contains app.css and the uploads/ folder for runtime resume storage

app.py is minimal and only handles app configuration, DB initialisation, the ensure_schema call, upload folder setup, and Blueprint registration. Role-wise controllers are split under route/ (auth, admin, company, student, public) to keep code maintainable. All data models are in model.py, and static assets are under static/.

Implemented features include role-based login and registration, company approval workflow, placement drive posting and moderation, student drive browsing with active-only and CGPA-based filtering, duplicate application prevention via a unique index on (student_id, drive_id), full application status

pipeline (applied -> shortlisted -> placed/rejected/withdrawn), student and company notification views powered by the shared Log table, analytics dashboards with Chart.js pie and bar charts across all three roles, admin search for students and companies, blacklist/activate for users with auto-closure of company drives, and schema auto-migration support for older local database files.

8. Video Presentation

Drive Link:

 Superset-Demo-Video.mp4

or

https://drive.google.com/file/d/1MleuMywdDzKULMOYaTbwY21MIT_M_9iU/view?usp=sharing